



wanted to build a Claude bot to find opportunity in prediction markets, is this doable

Yes, it's doable, but you're competing with very optimized bots already running on markets like Polymarket and Kalshi, so you need to be realistic about edge, speed, and legal risk.^[1] ^[2]

What's already happening

- On Polymarket, high-frequency arbitrage and AI-driven bots already harvest mispricings and latency edges, making thousands of micro-trades with smooth, linear P&L.^[1]
- Cross-platform arbitrage (e.g., Polymarket vs Kalshi) is a known strategy, exploiting small price gaps on binary event contracts, especially geopolitical and macro events.^[2]
- There are people publicly moving from "monitoring a dashboard" to near-atomic execution bots that automatically hit both sides of an arb spread.^[3]

Where a Claude-style bot fits

A Claude-like agent can help in three main ways:

- **Idea generation & research:** scanning news, social data, and market listings to surface candidate markets where odds look off relative to real-world probabilities.^[2] ^[1]
- Strategy orchestration: acting as a coordinator between specialized components (data feed, pricing model, execution engine), similar to multi-agent LLM trading frameworks in stocks.^[4] ^[5]
- Risk and ops automation: managing position limits, pausing on abnormal volatility, and explaining what the system is doing in plain language, which current low-level bots don't do well.^[6] ^[4]

You would still need non-LLM components for fast pricing, order routing, and latency-sensitive arbitrage; the LLM is more of a **brain** around those systems than the execution core.^[7] ^[4]

Constraints and challenges

- Market efficiency: prediction markets saw big volume growth through 2025 and are increasingly "institutional," so pure arb windows are smaller and shorter-lived.^[2]
- Speed: profitable arb often requires near-simultaneous fills on both legs; even a minute's delay can flip a profitable spread into a loss.^[3] ^[2]
- Legal gray area in Texas: sports betting and most gambling are still technically illegal, and prediction markets sit in a loophole that Texas hasn't clearly blessed, especially for sports-style contracts.^[8] ^[9] ^[10]

- Model limits: papers on LLM trading show that even when an LLM predicts market direction, actual trading profitability can lag because of slippage, fees, and noise. ^[7]

Rough architecture you'd need

At a high level, a practical system looks like:

- Market data layer: real-time APIs/WebSockets to one or more prediction markets (quotes, order books, your positions). ^[2]
- Pricing/edge engine: rule-based or ML models that convert event information into implied probabilities, compare to market odds, and flag edges above a threshold.
- Execution engine: fast, non-LLM code to place, cancel, and update orders; handle partial fills; and attempt atomic-like execution for arb legs. ^{[6] [3]}
- Claude agent:
 - Monitors feeds and logs, decides which strategies to enable/disable, tunes thresholds.
 - Explains current risk, big positions, and P&L in natural language.
 - Helps you iterate on new strategies and sanity-checks changes before deployment. ^[5]^[4]

Practical next steps

If you want to pursue this, a realistic path is:

- Start with semi-automated: human-in-the-loop system where Claude surfaces candidate trades and you click to execute.
- Focus first on **mispricing detection** and risk dashboards, then gradually automate execution once you've validated edge and failure modes.
- Consult a Texas attorney about using offshore or federally regulated platforms from Houston, especially if you plan to scale size or take outside capital. ^{[9] [10] [8]}

If you share which platforms you're targeting (Polymarket, Kalshi, etc.) and your dev stack, I can outline a more concrete blueprint (modules, APIs, and what to delegate to Claude vs hard-coded logic).



poly market and kalshi, want to delegate it all to Claude

You can use Claude to coordinate and research trades on Polymarket and Kalshi, but you should not delegate the entire, fully-automated system to an LLM alone—execution, risk, and custody must be hard-coded infrastructure around it. ^{[11] [12] [13]}

What “delegate it all to Claude” can reasonably mean

Claude can reliably handle:

- Market scanning: querying Polymarket and Kalshi APIs for new and existing markets, filtering by volume, fees, spreads, and open interest, then surfacing candidate opportunities. ^[14] ^[12] ^[11]
- Edge detection logic design: generating and refining the rules or models that compute implied probabilities, sum-to-one arbitrage, cross-market spreads, and thresholds, which you then implement in code. ^[15] ^[11]
- Strategy orchestration: deciding which strategies are “on,” adjusting risk parameters, and producing natural-language explanations of positions, P&L, and recent trades. ^[13] ^[16]
- Research and context: digesting news or data relevant to specific event markets (e.g., macro prints, elections, weather) and mapping that to contracts on Kalshi and Polymarket. ^[17] ^[14] ^[13]

In practice, Claude becomes the **control layer** and research brain, but not the low-level trading engine. ^[12] ^[11] ^[13]

What must stay outside Claude

For Polymarket and Kalshi you need a conventional bot stack that Claude *steers* instead of directly replacing:

- API and data layer
 - Polymarket: use Gamma API for markets and metadata, CLOB API plus WebSockets for order book and trades, with very low-latency handling. ^[11]
 - Kalshi: use their REST and (optionally) FIX-style APIs for market data, order management, and portfolio tracking. ^[14] ^[12]
- Execution engine
 - Handles WebSocket subscriptions, order placement, cancels, FOK/IOC orders, and re-quoting without waiting on LLM responses. ^[18] ^[11]
 - Implements concrete strategies like sum-to-one arbitrage ($YES+NO < 1$), intra-market combos, and cross-market spreads that clear fees (e.g., $>2.5-3\%$ on Polymarket given 2% fee). ^[15] ^[11]
- Risk and security
 - Position limits per market and globally, max exposure per outcome, stop-trading switches, and key management all live in code you control.
 - LLM suggestions must be validated against these hard constraints before any order is actually sent.

This mirrors how successful Polymarket bots are built today: a fast, modular bot with data collector, strategy engine, order manager, and risk manager, with any AI sitting on top instead of in the hot path. ^[19] ^[11] ^[15]

Example architecture with Claude as “boss”

A realistic setup for Polymarket + Kalshi automation might look like:

- Core bot services (your code)
 - MarketDataService: subscribes to Polymarket CLOB and Kalshi feeds, normalizes order books and trades. [\[12\]](#) [\[11\]](#)
 - StrategyService: implements specific rules (e.g., buy both Polymarket YES/NO if combined price < 0.97 after fees; take cross-market arb if Kalshi “Yes” is 0.60 and Polymarket equivalent is 0.50 with sufficient liquidity). [\[11\]](#) [\[15\]](#)
 - ExecutionService: places and cancels orders, uses FOK for arb legs, monitors fills, and avoids dangling one-sided risk. [\[18\]](#) [\[11\]](#)
 - RiskService: enforces exposure caps, daily loss limits, and circuit breakers.
- Claude agent
 - Receives summarized state: top opportunities, open positions, recent P&L, violations/alerts.
 - Decides high-level actions: e.g., “increase threshold on cross-exchange arb to $\geq 4\%$,” “pause BTC-minute markets after abnormal slippage,” “focus on Kalshi macro prints this week.” [\[13\]](#) [\[14\]](#)
 - Generates updated config: strategy parameters, whitelisted markets, and risk caps that your services read and enforce.
 - Explains behavior: “We are long X on Polymarket and short X on Kalshi in market Y because spread is Z and fees are W.”

This keeps Claude in a supervisory role while the time-critical work is robust, testable code.

Why you shouldn’t fully hand the wheel to Claude

- Latency and determinism: profitable Polymarket bots are exploiting brief mispricings and exchange lags, with thousands of small, precise trades—this demands deterministic, low-latency code that an LLM can’t guarantee in real time. [\[20\]](#) [\[13\]](#) [\[11\]](#)
- Safety: giving an LLM direct write access to keys and uncapped order placement is a recipe for catastrophic mistakes; even small prompt or context issues can trigger wild positions.
- Competition: the top Polymarket bots have already pulled tens of millions in arbitrage; they combine engineered infra plus narrow strategies, not “fully autonomous” LLMs. [\[20\]](#) [\[15\]](#) [\[11\]](#)

How to move toward maximum automation

If your goal is “as close to fully delegated as safely possible,” a staged path is:

1. Advisory mode
 - Claude reads snapshots/logs and suggests trades; you click to approve.
2. Guard-railed auto mode

- Claude proposes trade instructions in a strict schema; your engine validates them against rules and executes only if safe.

3. Strategy and config automation

- Execution is fully automated, but Claude dynamically tunes which strategies are active and what parameters they use, within fixed bounds.

If you tell me what language/stack you're comfortable with (e.g., Python on a VPS) and your risk tolerance, I can sketch a more concrete module-by-module blueprint for your Polymarket + Kalshi "Claude-as-boss" setup.



im thinking like whenever I check in if it can provide me a really good opportunity that's mispriced and I take the trade

That use case is very doable: think of Claude as a signal engine that constantly scans Polymarket + Kalshi and, when you ask, shows you the best current mispricings for you to click and trade. [\[21\]](#) [\[22\]](#)

What the system would actually do

- Continuously pull markets and order books from Polymarket (Gamma + CLOB APIs) and Kalshi, normalize them, and track equivalent events. [\[22\]](#) [\[21\]](#)
- Compute implied probabilities, check sum-to-one within each market, and look for cross-venue gaps where the same event is priced materially differently after fees. [\[21\]](#) [\[22\]](#)
- Maintain a ranked list of "opportunities" with size, spread after fees, liquidity, and resolution risks, so when you check in, Claude can surface the 1–3 highest-quality ideas with reasoning. [\[23\]](#) [\[24\]](#) [\[25\]](#)

In practice, this behaves like a custom, AI-driven Polymarket/Kalshi dashboard: you stay fully in control of execution, but you don't have to manually scan hundreds of markets each time. [\[26\]](#) [\[27\]](#) [\[25\]](#)

Key pieces you'd need

- Data/infra:
 - A small service that streams prices and order books from Polymarket's Gamma + CLOB WebSocket and Kalshi's API, similar to beginner bot setups. [\[28\]](#) [\[21\]](#)
 - Logic to map "same" events across venues and handle different resolution sources (e.g., CFTC vs on-chain oracle), which is where some arbs and risks arise. [\[29\]](#) [\[23\]](#) [\[22\]](#)
- Strategy logic:
 - Rules for what counts as "good": minimum edge (e.g., $\geq 3-4\%$ after fees), minimum depth on each side, your max size, and a simple risk score for resolution differences. [\[23\]](#) [\[22\]](#)

- Optionally, tags for the kinds of trades you like (elections, macro, sports) so the system prioritizes those.
- Claude layer:
 - Reads a summarized snapshot of opportunities and your preferences, then outputs a short list: "Here are the top 2 mispricings right now, with expected edge, size, and main risks."
 - Lets you ask follow-ups like "only show stuff that settles in <2 weeks" or "only US-macro markets."

How your workflow would look

- You open your interface (CLI, web, or chat) and ask: "Any good Polymarket vs Kalshi mispricings right now?"
- Claude responds with:
 - Trade idea A (details, edge, why it exists, resolution caveats).
 - Trade idea B (same, possibly in a different theme).
- You manually hit "trade" on your own accounts or via a separate minimal execution button; the system never places orders without you.

If you tell me your rough budget, favorite contract types (elections vs macro vs sports), and tech comfort level (no-code, Python, etc.), I can sketch a concrete minimal v1 for this signal-only setup.

✱

1. <https://finance.yahoo.com/news/arbitrage-bots-dominate-polymarket-millions-100000888.html>
2. <https://www.ainvest.com/news/geopolitical-event-trading-prediction-market-arbitrage-2025-investment-analysis-2601/>
3. https://www.reddit.com/r/algotrading/comments/1qbq7j1/automating_the_prediction_market_arb/
4. <https://tradingagents-ai.github.io>
5. <https://arxiv.org/abs/2506.16813>
6. <https://pixelplex.io/blog/crypto-arbitrage-bot-development/>
7. <https://neurips.cc/virtual/2025/132549>
8. <https://www.click2houston.com/news/local/2025/09/16/houstonians-cashing-in-on-loopholes-in-laws-that-allow-sports-betting-in-texas/>
9. <https://woai.iheart.com/featured/houston-texas-news/content/2026-01-14-texas-sits-out-on-major-sports-betting-lawsuit/>
10. <https://www.covers.com/industry/the-year-ahead-in-texas-sports-betting-bettors-gamble-on-prediction-markets-dec-24-2025>
11. <https://www.quantvps.com/blog/automated-trading-polymarket>
12. <https://zuplo.com/learning-center/kalshi-api>
13. <https://finance.yahoo.com/news/arbitrage-bots-dominate-polymarket-millions-100000888.html>
14. <https://www.panewslab.com/en/articles/7v5h2okw>

15. <https://www.quantvps.com/blog/cross-market-arbitrage-polymarket>
16. <https://defiprime.com/definitive-guide-to-the-polymarket-ecosystem>
17. <https://solana.com/news/dflow-prediction-markets-api>
18. https://www.reddit.com/r/PolymarketTrading/comments/1qnsm2d/trading_bots_where_to_get_started/
19. <https://github.com/zydomus219/Polymarket-betting-bot>
20. <https://www.ainvest.com/news/arbitrage-bots-dominate-polymarket-millions-profits-humans-fall-2601/>
21. <https://www.quantvps.com/blog/setup-polymarket-trading-bot>
22. <https://www.quantvps.com/blog/cross-market-arbitrage-polymarket>
23. <https://defiprime.com/definitive-guide-to-the-polymarket-ecosystem>
24. <https://www.mexc.co/news/457778>
25. <https://whales.market/blog/top-10-useful-tools-on-polymarket-part-1/>
26. <https://blacklabel.net/blog/business-insights/dashboard-alerts-catch-key-trading-signals-quickly/>
27. <https://www.yellowfinbi.com/suite/signals>
28. <https://zuplo.com/learning-center/kalshi-api>
29. https://www.reddit.com/r/algotrading/comments/1q83w3d/found_5_arbitrage_spreads_in_prediction_markets/
30. <https://newyorkcityservers.com/blog/best-prediction-market-bots-tools>
31. <https://sparkco.ai/blog/us-gdp-growth-surprise-prediction-markets>